Week 11 : Behavior Cloning

1. Introduction

In this practice session, we will use behavior cloning, one of the imitation learning methods, to train an RC Car agent in a simulation environment. Behavior Cloning (BC) is a type of imitation learning where an agent learns to replicate the behavior of an expert by mimicking the actions taken by the expert in given states. The goal is to train the agent so that its policy matches the expert's policy as closely as possible. After training, we will deploy the trained model onto a real vehicle to conduct a sim-to-real experiment on a track. In this session, students should aim to understand imitation learning, write code, and train an agent to achieve good performance. This instruction will provide a background of the behavior cloning algorithm with sample code details.

2. Background

2.1. Imitation learning

Imitation learning is a significant framework in robot learning where an agent learns to replicate observed expert behaviors without relying on an explicitly defined reward function. A key approach within this framework is Inverse Reinforcement Learning (IRL), which focuses on inferring the underlying reward function that likely drives the observed behaviors. Once this reward function is estimated, the agent can optimize a policy that maximizes the inferred reward, effectively learning the behavior in a more generalizable manner. In contrast, Behavior Cloning (BC) maps observed behaviors to actions through supervised learning, creating a policy that directly imitates the demonstrator's actions.

2.2. Deep learning

Deep learning is an optimization technique that focuses on training neural networks with multiple layers to model and learn complex patterns in data. By employing methods like gradient descent, deep learning iteratively adjusts the network's parameters to minimize a loss function, thereby optimizing the model's performance. This approach enables deep learning models to capture intricate relationships and representations within large datasets, making them particularly effective for tasks such as image and speech recognition, natural language processing, and other areas where traditional machine learning methods may fall short.

3. Code Implementation

In this instruction, we will give you a guide for the implementation of behavior cloning algorithm.

1. Base code

- Base agent (common/agent_base.py) - DO NOT CHANGE

The base agent code is provided and should not be modified. This file contains essential functions that define the agent's interaction with the environment. The main task for this session is to implement the *train()* function in *imitation/agent.py*. This function is where the core algorithm will be coded.

- Model architectures (common/models.py)

The skeleton code for the actor network is defined in this file. Your task is to implement the architecture. Focus on the "*implement here*" sections to define the specific layers and forward pass logic.

✓ **TIPS**: You can use simple linear layers with *Actor*, instead of *SquashedActor*.

2. Agent code

- **Step 1: Collect Trajectories** (main_il.py)

```
# ======= collect trajectories ======= #
expert_data_path = 'expert_data.pkl'
if os.path.isfile(expert_data_path):
    cprint(f'\n[Rollout] load.', bold=True, underline=True)
    with open(expert_data_path, 'rb') as f:
        expert_data = pickle.load(f)
        observations = expert_data['observations']
        actions = expert_data['actions']
else:
    cprint(f'\n[Rollout] start.', bold=True, underline=True)
    total_step = 0
    observations, actions = [], []
    while total_step < args.total_steps:

        observation, info = train_env.reset()
        track = train_env._env.unwrapped.track
        planner = PurePursuitPlanner(track=track, wb=0.17145 + 0.15875)
        ep_observations, ep_actions = [], []
        ep_step, ep_reward = 0, 0

        terminate = False
        while ep_step < args.max_episode_steps and not terminate:
            speed, steer = planner.plan(info['obs_dict'], PARAM)
            action = np.clip([steer/(train_env.max_steer+EPS), speed/train_env.max_speed], -np.ones(2), np.ones(2))
            ep_observations.append(observation)
            ep_actions.append(action)

            observation, reward, terminate, truncate, info = train_env.step(action)
            ep_step += 1
            ep_reward += reward

        if ep_step > args.threshold:
            observations.append(ep_observations)
            actions.append(ep_actions)
            total_step += ep_step
            print(f'[Total step] {total_step}: {ep_step}/{ep_reward:.2f}')

    observations = np.concatenate(observations, axis=0)
    actions = np.concatenate(actions, axis=0)
    with open('expert_data.pkl', 'wb') as f:
        pickle.dump({'observations':observations, 'actions':actions}, f)
agent.step(observations, actions)
train_env.close()
# ================================== #
```

For collecting expert data, we can use the pure pursuit controller with the *f1tenth* environment. After a predetermined number of steps (*total_steps*), the collected trajectories are used to update the agent. This involves storing the states and actions that will be used in the subsequent steps of the algorithm.

- **Step 2: Compute Actor Loss** (imitation/agent.py - train)

```
def train(self):
    states_tensor, actions_tensor = self.rollout_dataset.getBatches()
    action_dists = self.actor(states_tensor)

    # ============================ implement here ============================ #
    MSE_Loss =
    action_mean =
    actor_loss =
    # ======================================================================== #
```

We can train a model to predict the expert's actions given the states as input. The model will learn to map states to actions, effectively learning the expert's policy.

For loss function, the **Mean Squared Error (MSE)** loss is typically used in behavior cloning. It measures the average of the squares of the differences between the predicted actions by the model and the actual actions taken by the expert.

- ***Why Use MSE Loss?***

  MSE loss is chosen because it penalizes large errors more than small ones, making it suitable for regression tasks like Behavior Cloning, where we want the predicted actions to be as close as possible to the expert's actions.

$$\text{MSE Loss} = \frac{1}{N}\sum_{i=1}^{N}(\hat{a}_i - a_i)^2$$

Here, ***N*** is the number of samples, *a_hat* and *a_i* refer to the predicted actions and expert actions respectively.

✓ **TIPS**: You can use MSE loss from the pytorch module.

```
# update
self.optimizer.zero_grad()
actor_loss.backward()
torch.nn.utils.clip_grad_norm_(self.actor.parameters(), self.max_grad_norm)
self.optimizer.step()

train_results = {
    'actor_loss':actor_loss.item(),
    }

return train_results
```

After calculating the loss, perform gradient descent on this loss to update the actor network.